What's in a name?

Or what is wrong with eth0?

Stephen Hemminger

Microsoft Corporation
Redmond, WA USA
stephen@networkplumber.org

Abstract

Network interface names in Linux have started with *eth0* since the first introduction of TCP/IP in version 0.99[1]. But this convention is hard to support in modern hardware and cloud environments. Hardware may provide more information to help, and the emulation of that hardware in virtualized environments may make the problem worse.

Introduction

Network device names are the natural identifier interacting with the network subsystem and are used in many places from socket API's for binding to a device, ioctl's for managing a network device (ifreq), proc and sysfs filesystems for other management operations. Unlike most other devices in Linux, network devices do not have an entry in the normal device filesystem (/dev).

An additional constraint is that network device names are limited to 15 characters by the API constant IFNAMSIZ[2]. This value was inherited from early BSD Unix. Since this is in the kernel to userspace binary interface (ABI), it can not be increased.

Background

The first version of Linux networking had a table of possible devices and the first Ethernet device was assigned *eth0*, etc. This worked because their was no auto discovery mechanism on the early Industry Standard Architecture (ISA) bus and their was no Symmetric Multi Processing (SMP) support. When SMP processing and Peripheral Component Interconnect (PCI) bus was implemented this simple solution was no longer deterministic. This meant that names could be swapped on different reboots.

The earliest attempt at solving this problem in Linux was done in hotplug subsystem module (**udev)** by binding previously seen names to the Ethernet Media Access Control (MAC) address. This ensured that the same card would appear as *eth0* no matter which order it was discovered.

A better solution was invented by Dell with **biosdevname**[3]. The **biosdevname** program would create a network device name (such as *p2p1*) based on system information it discovered by examining the system PCI buss and Device Management Information (DMI). In practice, most devices ended up with names based on the PCI address; the DMI information was correct on a small number of systems. The methods used to extract the information were also problematic (direct access to kernel memory) so the kernel was enhanced to provide more information such as hot plug bus, ACPI, and slot information.

A more robust solution "Predictable network device names"[4] was developed as part of the wide ranging **systemd** project. The reimplementation of **udev** provides a mechanism for persistent naming based on the bus information provided through sysfs.

Device naming issues

Network devices names like other resources in the system should obey three fundamental properties in order to be managed.

Persistence

The name of a network device must be the same on each system boot. Many parts of the system initialization from assigning addresses and routes to important security features like firewall rules rely on having the same name on each system boot. If *eth1* and *eth0* appear at different locations each time networking won't work correctly.

Portability

When configuring multiple systems, the administrator wants all systems to have the same network names. It makes administering multiple systems much more difficult if each one has a different network name base on Ethernet MAC address, network card vendor or BIOS version. In a cloud environment, one image is deployed on 10 or even 1000's of machines.

The original **udev** mechanism had this problem. If a network card was replaced in a system, the default network name would change. With slot based naming (from systemd) this is not a problem.

An alternative model used on BSD systems is to assign network device names by network card vendor name. This also fails the portability requirement; if one system has an Intel 10Gbit Ethernet card then the name of the device

ixgbe0 but if instead it has a Qlogic card then it is named *qlxb0*. This makes copying configuration impossible.

Plausibility

The network device names should also be reasonable length and logical. The name should be as short as possible and relatively easy to manage.

Systemd/udev naming policy

Almost all modern Linux distributions use **systemd**. Even though each distribution seems to have a different mechanism for managing network configuration; most use the network naming policy from udev in systemd. Udev assigns names by a set of rules:

On board → eno1

Devices that are directly attached to the system (ie no bus).

PCI slot → ens3

PCI devices that support the hotplug API provide slot information.

PCI location → enp2s0

PCI address can also be used to form a name. But the address does vary between system vendors.

MAC address → enx78e7d1ea46da

For USB and some other devices, a name based on the MAC address is used.

None of the above → eth0

If no other info is available the name is left at original default.

Outstanding issues

When this works, it works well but many systems have problems.

Virtualized buses

Hypervisors provide emulated PCI bus. Devices on the bus maybe real hardware (pass through) or emulated in the hypervisor. The emulated bus often has quirky values for address and slot information. For example, VMWare provided PCI bus information which would lead to the unreasonable network name of *eno16777728.*

Bad ACPI information

Systemd relies on the kernel for its information, and the kernel relies on information provided by the ACPI tables in the BIOS. BIOS values are commonly broken and rarely can be fixed. An example was a system which had multiple Ethernet ports coming out of the back labeled 1, 2, 3, 4; but the BIOS numbering was 3, 2, 1, and the last port had information was missing.

SR-IOV

High speed network devices often support multiple virtual function (VF) devices through the PCI Single Root I/O vector (SR-IOV) standard. This allows a virtual instance of the device to be mapped into a guest operating system. The problem is that if the guest is migrated to another instance of the hypervisor, the PCI bus information maybe completely different on the migrated host. Therefore the VF device may change names when migrated.

Link Aggregation

A common solution to VF migration is to use Link Aggregation (also known as bonding or teaming) to join a high speed VF device with a lower speed virtual network interface[5]. The resulting device is named *bond0* (or *team0* if using teaming) in Linux. This can still cause confusion since it is different from other devices in the system and the underlying devices are still visible.

Other meta-data

In addition to name, network devices have other information associated with them. These can help management since they have different characteristics.

Ifindex

Each network device has a unique non-zero numeric value. This device is not guaranteed to be the same across reboot and can be reassigned when device is removed. But the ifindex remains the same even if device is renamed. Therefore a correctly written application should lookup the name to index mapping once, and use the index for the rest of the control operations. This avoids any race conditions with renaming.

IfAlias

A network device may also have an alias assigned to it. This value is not unique but is commonly used to describe what it is connected to (for example “Corporate backbone”). This name can be much longer (255 characters) and does not have to be unique.

This is not the same as the (now deprecated) Linux network aliases that used to be used to assign multiple addresses to the same device (ie. eth0:1).

IfDescr

SNMP also has a descriptive string. With the commonly used SNMP daemon on Linux this is filled in with the information extracted from the PCI vendor database (i.e. “Intel Corporation 82559 Ethernet Controller Virtual Function (rev 01)”).

Recommendations

Eth0 Standard

The network device *eth0* is now the defacto standard in the cloud. Amazon Web Services, Microsoft Azure, and Docker containers reserve it as the primary or management network device.

Better handling of failover

The setup of link aggregation (via bonding or team) for migration failover is done by ad-hoc scripts. This needs to be better supported either by common tools such as network manager or teaming daemon. This would resolve issues such as how to pair the primary (VF) device with the secondary (synthetic) device.

Hiding slaves

When link aggregation is used to provide failover, the subsidiary network devices are still visible in the system. This maybe useful for diagnosing state transition issues but exposes more devices that may confuse or clutter management of devices by applications. Suggestions have been made to use network namespaces, flags, or special naming conventions to solve this problem.

Host bridge alias info

In virtualized environments, the system host hypervisor has its own configuration infrastructure. This configuration infrastructure already contains topology information about how each virtual switch is connected. At present, there is no mechanism to propagate the information back to the guest. For example, the hypervisor may know that the guest *eth0* is connected to virtual switch that is connected to “Test backbone network”. This information could then be used to set network interface alias value and be visible to tools.

Conclusion

Persistent, portable and plausible network naming is a hard problem that is not fully solved. The current model does a good job for physical systems with well supported infrastructure. But new features and infrastructure are still necessary to improve the user experience in cloud environments.

Acknowledgments

I would like to thank the Linux user community for their patience and feedback in the face of ever changing network naming. Also, the systemd developers for taking on the problem, even in the face of user complaints. Lastly, thank you for the contributions of so many developers of Linux for building the most complete network operating system available.

Author Bibliography

Stephen Hemminger is a software engineer at Microsoft since 2016. He has worked on TCP congestion control, network device management, routing, VXLAN, and many other parts of Linux networking.

References

1. First reference to eth0:
   Torvalds, Linus. "Linux-0.98." Public Git Hosting - davej-history.git/commit. September 29, 1992. Accessed April 8, 2017. http://repo.or.cz/davej-history.git/commit/e25f534048a8e0069268fe14e2d2f528effa22f4
2. Stevens, W. Richard. UNIX network programming. Upper Saddle River, NJ: Prentice Hall PTR, 1998.
3. Narendra K. "Consistent Network Device Naming in Linux." July 2012. Accessed April 8, 2017. https://linux.dell.com/files/whitepapers/consistent_network_device_naming_in_linux.pdf.
4. Rayhawk, Joe. "Predictable Network Interface Names." PredictableNetworkInterfaceNames. May 18, 2013. Accessed April 8, 2017. https://www.freedesktop.org/wiki/Software/systemd/PredictableNetworkInterfaceNames/.
5. Hudek, Ted. "SR-IOV VF Failover and Live Migration Support." SR-IOV VF Failover and Live Migration Support. September 2, 2016. Accessed April 8, 2017. https://msdn.microsoft.com/en-us/windows/hardware/drivers/network/sr-iov-vf-failover-and-live-migration-support.